

Solution Architecture

Date	17 September 2026
Team ID	SWTID-2026-2311
Project Name	AI StudyBuddy API
Maximum Marks	4 Marks

Solution Architecture - Layered Component View

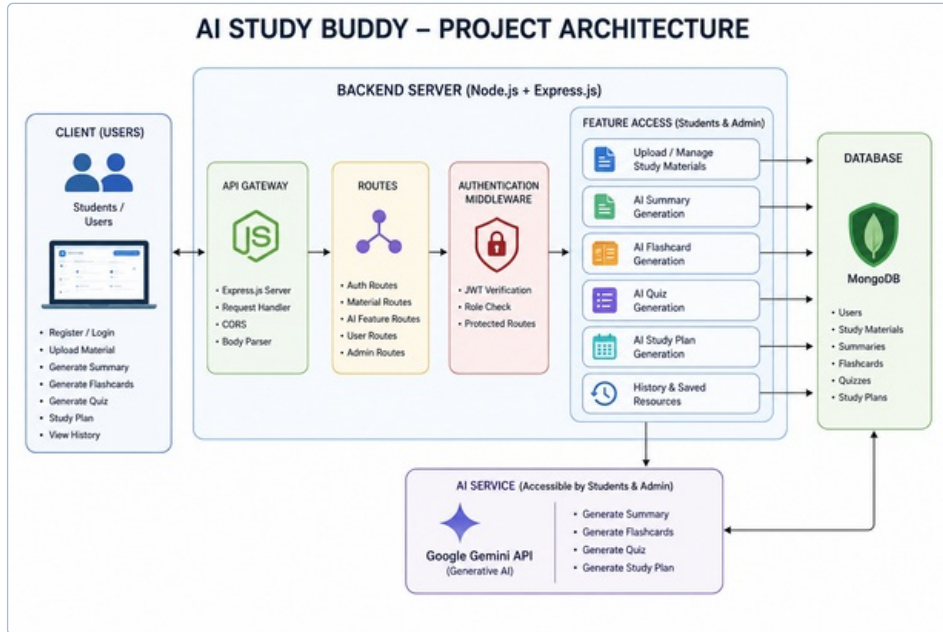


Fig. 1 - AI StudyBuddy layered architecture, as designed by the team

Component Description

Layer	Responsibility
Client Application	React.js frontend for register/login, uploading study material, and viewing AI-generated summaries, flashcards, quizzes and study plans through the REST API.
Express Server	<code>index.js</code> bootstraps Express, loads <code>dotenv</code> , applies <code>cors</code> and JSON body-parsing, and mounts the <code>/api/auth</code> , <code>/api/materials</code> and <code>/api/admin</code> route groups.
Authentication Middleware	<code>src/middleware/auth.js</code> exports <code>protect</code> (verifies the Bearer JWT with <code>JWT_ACCESS_SECRET</code> and attaches <code>req.user</code>) and <code>adminOnly</code> (403s any non-admin role) for role-based route protection.
Route Handlers	<code>routes/auth.js</code> , <code>routes/materials.js</code> , <code>routes/admin.js</code> declare each endpoint's path/method; all study-material and AI-generation endpoints (summary, flashcards, quiz, study plan) are nested under the materials route group.
Controller Layer	<code>controllers/authController.js</code> , <code>materialController.js</code> , <code>adminController.js</code> hold request-handling logic: validation, calling models/utilities, and shaping the JSON response.
AI Service Layer	<code>src/utlis/gemini.js</code> isolates all Google Gemini calls behind a single <code>askGemini(prompt)</code> function, so controllers stay free of AI-vendor-specific code.
Database Layer	<code>models/User.js</code> and <code>models/Material.js</code> (Mongoose schemas) plus <code>src/utlis/db.js</code> (MongoDB Atlas connection) persist and validate all data.

MVC Design Pattern

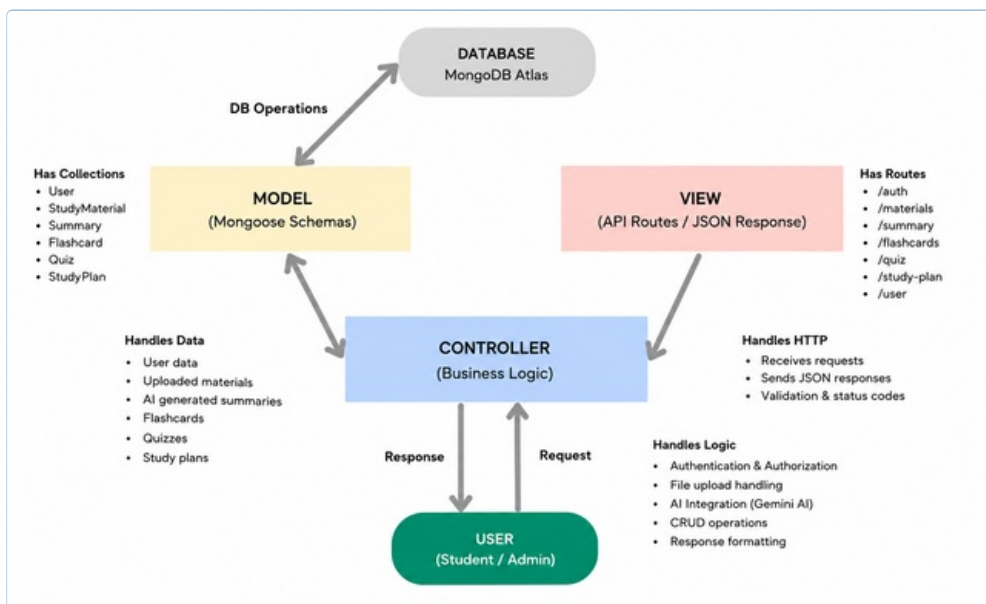


Fig. 2 - MVC mapping used across the codebase (Database: MongoDB Atlas)

Models (`User` , `Material`) define schema and validation; Controllers handle authentication/authorization, file-upload handling, Gemini AI integration, CRUD operations and response formatting; there is no server-rendered View layer since the project is a pure JSON REST API - the "View" is the JSON response itself, returned over the `/auth` , `/materials` , `/summary` , `/flashcards` , `/quiz` , `/study-plan` and `/user` routes to the React client or any API tool.